TECHNICKÝ PRŮVODCE · HTML / CSS / JAVASCRIPT

Jak je aplikace zakódovaná

Struktura projektu, data a výběr náhodných otázek

1 Technologie a soubory

HTML vytváří strukturu, CSS vzhled a JavaScript herní logiku. Aplikace běží přímo v prohlížeči bez serveru a externích knihoven.

```
<link rel="stylesheet" href="styles.css">
<script src="script.js" defer></script>
```

index.html = rozhraní · **styles.css** = vzhled · **script.js** = otázky, události a pravidla. Atribut defer spustí skript až po načtení HTML.

2 Datová struktura otázky

Question bank je objekt s poli otázek pro easy, medium a hard. Každá otázka obsahuje kategorii, zadání, čtyři odpovědi a index správné odpovědi.

```
{
  category: "Věda",
  question: "Který chemický prvek má značku Fe?",
  answers: ["Fluor", "Fosfor", "Železo", "Francium"],
  correct: 2
}
```

Index `correct` začíná nulou: hodnota 2 označuje třetí prvek pole odpovědí.

3 Míchání: Fisherův–Yatesův algoritmus

Funkce `shuffled(items)` vytvoří kopii pole a postupně náhodně prohazuje prvky. Banku otázek tedy nemění.

```
function shuffled(items) {
  const copy = [...items];
  for (let index = copy.length - 1; index > 0; index -= 1) {
    const swapIndex = Math.floor(Math.random() * (index + 1));
    [copy[index], copy[swapIndex]] =
      [copy[swapIndex], copy[index]];
  }
  return copy;
}
```

4 Výběr bez opakování

`selectQuestions(level)` filtruje otázky, jejichž text už je uložený v `Set` dané obtížnosti. Náhodně vybere požadovaný počet. Při vyčerpání sady historii resetuje a doplní zbývající místa.

```
const usedQuestions = {
  easy: new Set(), medium: new Set(), hard: new Set()
};
const unseen = shuffled(
  pool.filter(q => !used.has(q.question))
);
```

Časovač, odpovědi a body

Jak JavaScript vykresluje otázku a počítá výsledek

5 Stav hry a vykreslení

Proměnné `questions`, `questionIndex`, `score` a `correctAnswers` drží průběžný stav. `startQuiz()` stav vynuluje a zavolá `renderQuestion()`. Ta vytvoří HTML tlačítka pomocí DOM API a připojí ke každému událost kliknutí.

```
const button = document.createElement("button");
button.type = "button";
button.className = "answer";
button.addEventListener("click", () => selectAnswer(index));
answerContainer.append(button);
```

6 Časování otázky

`setInterval(..., 1000)` snižuje zobrazený čas jednou za sekundu. Po změně otázky se starý interval ruší přes `clearInterval()`. Přesný čas odpovědi se měří pomocí `performance.now()`, což umožňuje správně rozhodnout o bonusu i mezi tiky časovače.

```
timerId = setInterval(() => {
  secondsLeft -= 1;
  if (secondsLeft <= 0) {
    clearInterval(timerId);
    selectAnswer(-1); // čas vypršel
  }
}, 1000);
```

7 Kontrola odpovědi a bodování

`selectAnswer()` hlídá příznak `answered`, aby nešlo tutéž otázku započítat vícekrát. Porovná vybraný index s indexem správné odpovědi. Správná odpověď má základ 100 bodů a rychlostní bonus přidá dalších 100.

```
const speedThreshold = difficulty === "hard" ? 5 : 3;
const speedBonus = elapsedSeconds <= speedThreshold;
const earned = speedBonus ? 200 : 100;
score += earned;
```

Odpovědi se následně zamknou a správná/špatná možnost se označí CSS třídou.

8 Výpočet výsledku a vzdání

`showResults()` spočítá procenta: počet správných odpovědí dělený počtem hodnocených otázek krát 100. Při vzdání se jako jmenovatel použije jen počet již zodpovězených otázek; nulový počet je zvlášť ošetřen.

```
const resultQuestionCount = abandoned
  ? answeredQuestions
  : questions.length;
const percentage = resultQuestionCount === 0 ? 0
  : Math.round(correctAnswers / resultQuestionCount * 100);
```

Ukládání, CSS a testování

Jak se mění motiv a uchovávají uživatelské volby

9 Nastavení v localStorage

Výchozí volby jsou v objektu `DEFAULT_SETTINGS`. Při startu se data načtou přes `localStorage.getItem()` a `JSON.parse()`; při změně je `JSON.stringify()` převede na text a `setItem()` uloží. Try/catch ošetřuje nedostupné úložiště.

```
function saveSettings() {
  try {
    localStorage.setItem(
      SETTINGS_KEY, JSON.stringify(settings)
    );
  } catch (error) {
    console.warn("Nastavení se nepodařilo uložit.", error);
  }
}
```

`applySettings()` přidá atributy `data-theme` a `data-accent` na body. `applyLanguage()` projde prvky s `data-i18n` a vloží překlad ze slovníku.

10 Události a zvuk

`addEventListener()` reaguje na kliknutí, změnu selectu a checkboxu. Funkce `updateSetting()` aplikuje změnu a uloží ji. Volitelný zvuk generuje Web Audio API: `AudioContext`, oscilátor `createOscillator()` a hlasitost `createGain()`.

11 CSS proměnné a responzivita

Barvy jsou definované jako CSS custom properties, například `--background` a `--purple`. Selektory `body[data-theme="light"]` nebo `body[data-accent="blue"]` změní proměnné podle nastavení. Media query `@media (max-width: 520px)` upraví rozložení pro mobil. `prefers-reduced-motion` omezí animace pro uživatele, kteří preferují méně pohybu.

12 Spuštění a kontrola

Projekt se otevře přes `index.html`. Syntaxi skriptu lze zkontrolovat příkazem:

```
node --check script.js
```

V prohlížeči se ověřil celý průchod hrou, bodování, časovač, vzdání, různé délky kola, nastavení, obnovení stránky i mobilní rozložení.

> **Hlavní logické funkce:** `shuffled()`, `selectQuestions()`, `startQuiz()`, `renderQuestion()`, `selectAnswer()`, `showResults()`, `applySettings()` a `saveSettings()`.